

컴퓨터 프로그래밍 3: Term 프로젝트

1. 프로젝트 개요

이번 프로젝트에서는 C 언어로 Mini Student Shell 을 구현한다.

이 프로그램은 학생 정보를 linked list 로 관리하고, 데이터를 CSV 파일에 저장한다. 사용자는 list, add, delete, update, find, save 와 같은 shell 형태의 명령어를 입력하여 학생 정보를 조회하거나 수정할 수 있다.

2. 주요 요구사항

1. 프로그램 시작 시 CSV 파일명을 command-line argument 로 입력받는다.
2. 시작 시 해당 CSV 파일에서 학생 정보를 읽어온다.
3. 학생 정보를 linked list 에 저장한다.
4. 명령어를 통해 학생 정보 추가, 삭제, 수정, 검색, 출력 기능을 제공한다.
5. Admin Program 과 Client Program 을 각각 빌드할 수 있도록 Makefile 을 제공한다.
6. Makefile 에서는 preprocessing flag 를 전달하여 Admin/Client Program 에 필요한 코드를 선택적으로 포함한다.
7. save 명령어를 통해 변경된 학생 정보를 같은 CSV 파일에 다시 저장한다.
8. 선택적으로 명령어 파일을 입력 받아 여러 명령어를 순서대로 실행한다.
9. 잘못된 명령어, 잘못된 입력, 파일 오류, 메모리 오류 등을 안전하게 처리한다.

3. 프로그램 실행 방법

프로그램은 시작할 때 CSV 파일명을 필수 인자로 받아야 한다.

```
./admin_shell students.csv
./client_shell students.csv
```

파일명이 주어지지 않은 경우에는 사용법을 출력해야 한다.

```
Usage: ./admin_shell <csv_file> [-f command_file]
Usage: ./client_shell <csv_file> [-f command_file]
```

프로그램 시작 시, 주어진 CSV 파일에서 학생 정보를 읽어온다.

```
[Admin Program]
Loaded 3 students from students.csv.
admin>
```

3.1 명령어 파일 옵션

프로그램은 선택적으로 명령어 파일을 옵션으로 받을 수 있다.

```
./admin_shell students.csv -f commands.txt
./client_shell students.csv -f commands.txt
```

명령어 파일에는 한 줄에 하나의 명령어가 들어 있다.

```
list
add 4 David 88
update 2 95
find 4
save
exit
```

프로그램은 CSV 파일을 먼저 불러온 뒤, 명령어 파일에 있는 명령어들을 위에서 아래로 순서대로 실행한다.

명령어 파일 실행 중 특정 줄에서 오류가 발생하면, 프로그램은 오류 메시지를 출력하고 다음 줄의 명령어를 계속 실행해야 한다.

```
[command file:1] list
ID   Name      Score
1    Alice     90
2    Bob       85

[command file:2] add 4 David 88
Student added.

[command file:3] update 99 70
Error: student not found. Skipped line 3.

[command file:4] find 4
ID: 4
Name: David
Score: 88
```

빈 줄은 무시할 수 있으며, #로 시작하는 줄은 주석으로 처리할 수 있다.

명령어 파일 실행이 끝난 뒤 exit 명령어가 실행되지 않았다면, 프로그램은 interactive shell 로 진입해도 되고 종료해도 된다. 단, 어떤 방식으로 동작하는지 README 에 명확히 설명해야 한다.

4. CSV 파일 형식

학생 정보는 CSV 파일로 저장한다. CSV 파일은 Excel 에서 바로 열어 확인할 수 있어야 한다.

```
id,name,score
1,Alice,90
2,Bob,85
3,Charlie,95
```

첫 번째 줄은 header 이다.

```
id,name,score
```

그 이후 각 줄은 학생 한 명의 정보를 나타낸다.

```
<id>,<name>,<score>
```

CSV 규칙

- id 는 정수여야 한다.
- name 은 비어 있을 수 없다.
- name 에는 쉼표를 사용할 수 없다.

- score 는 0 이상 100 이하의 정수여야 한다.
- 파일은 일반 텍스트 CSV 형식으로 저장해야 한다.

5. 자료구조 요구사항

학생 정보는 반드시 linked list 형태로 저장해야 한다.

```
typedef struct Student {
    int id;
    char name[32];
    int score;
    struct Student* next;
} Student;
```

6. 프로그램 모드

프로그램은 두 가지 실행 파일을 제공해야 한다.

- Admin Program
- Client Program

두 프로그램은 같은 소스 코드를 기반으로 작성하되, Makefile 에서 서로 다른 컴파일 플래그를 전달하여 preprocessing 단계에서 필요한 코드가 선택적으로 포함되도록 한다.

make admin 실행시 Admin Program (admin_shell) 빌드

make client 실행시 Client Program (client_shell) 빌드

6.1 Admin Program

Admin Program 은 모든 기능을 사용할 수 있는 관리자용 프로그램이다.

- 데이터 다시 불러오기
- 데이터 저장
- 학생 추가
- 학생 삭제
- 학생 점수 수정
- 학생 검색
- 전체 학생 출력
- 통계 확인
- 화면 지우기
- 프로그램 종료

./admin_shell students.csv

```
[Admin Program]
Loaded 3 students from students.csv.
admin>
```

6.2 Client Program

Client Program 은 조회 전용 프로그램이다. Client 사용자는 학생 정보를 수정할 수 없다.

- 데이터 다시 불러오기
- 학생 검색
- 전체 학생 출력
- 통계 확인
- 화면 지우기
- 프로그램 종료

Client Program 에서는 데이터 수정 명령어가 포함되지 않거나, 실행 시 권한 오류를 출력해야 한다.

```
./client_shell students.csv
[Client Program]
Loaded 3 students from students.csv.
client> add 4 David 88
Unknown command or permission denied.
```

6.3 Preprocessing 을 이용한 코드 선택

Admin Program 과 Client Program 은 preprocessing flag 를 이용해 필요한 코드만 선택적으로 포함할 수 있다.

```
#ifdef ADMIN_MODE
    // Admin Program 에서만 필요한 코드
#endif

#ifdef CLIENT_MODE
    // Client Program 에서만 필요한 코드
#endif
```

Makefile 에서는 -DADMIN_MODE, -DCLIENT_MODE 와 같은 플래그를 전달한다.

```
admin:
    $(CC) $(CFLAGS) -DADMIN_MODE $(SRCS) -o admin_shell

client:
    $(CC) $(CFLAGS) -DCLIENT_MODE $(SRCS) -o client_shell
```

7. 필수 명령어

프로그램은 다음 명령어를 지원해야 한다.

명령어	Admin Program	Client Program	설명
save	O	X	현재 linked list 를 원래 CSV 파일에 저장
reload	O	O	원래 CSV 파일에서 학생 정보를 다시 불러오기

add <id> <name> <score>	O	X	학생 추가
delete <id>	O	X	ID 로 학생 삭제
update <id> <score>	O	X	학생 점수 수정
find <id>	O	O	ID 로 학생 검색
list	O	O	전체 학생 출력
stats	O	O	학생 수, 평균, 최고점, 최저점 출력
help	O	O	사용 가능한 명령어 출력
clear	O	O	화면 지우기
exit	O	O	프로그램 종료

8. 명령어 상세 설명

8.1 save

현재 linked list 에 저장된 학생 정보를 원래 CSV 파일에 저장한다.

사용법:

```
save
```

예:

```
admin> save
Saved 4 students to students.csv.
```

Client Program 에서는 사용할 수 없다.

8.2 reload

원래 CSV 파일에서 학생 정보를 다시 불러온다.

사용법:

```
reload
```

예:

```
admin> reload
Reloaded 3 students from students.csv.
```

권장 동작: 현재 linked list 를 모두 해제한 뒤 CSV 파일을 다시 읽어 linked list 를 새로 구성한다.

8.3 add

새 학생 정보를 추가한다.

사용법:

```
add <id> <name> <score>
```

예:

```
admin> add 4 David 88
Student added.
```

가능한 오류: 인자 부족, 잘못된 점수, 점수 범위 오류, 중복 ID, 잘못된 이름.

에러 출력 문자열 요구사항 (채점 기준):

중복 ID → "duplicate" 또는 "Duplicate" 포함 메시지 (예: Error: duplicate ID.)

인자 부족 → "Error" 또는 "missing" 포함 메시지

그 외 잘못된 입력 (비숫자/0/음수 ID, 점수 범위 초과, 비숫자 점수) → "Error" 포함 메시지

8.4 delete

ID 를 기준으로 학생 정보를 삭제한다.

사용법:

```
delete <id>
```

예:

```
admin> delete 2
Student deleted.
```

해당 학생이 없을 경우 Error: student not found.

8.5 update

ID 를 기준으로 학생의 점수를 수정한다.

사용법:

```
update <id> <score>
```

예:

```
admin> update 2 95
Student updated.
```

가능한 오류: 학생 없음, 잘못된 점수, 점수 범위 오류.

해당 학생이 없을 경우 Error: student not found.

8.6 find

ID 를 기준으로 학생 정보를 검색한다.

사용법:

```
find <id>
```

예:

```
admin> find 1
ID: 1
Name: Alice
Score: 90
```

해당 학생이 없을 경우 Error: student not found.

8.7 list

linked list 에 저장된 모든 학생을 출력한다.

사용법:

```
list
```

예:

```
admin> list
ID   Name      Score
1    Alice     90
2    Bob       85
3    Charlie   95
```

학생 정보가 없을 경우 No students found.

8.8 stats

간단한 통계를 출력한다.

사용법:

```
stats
```

예:

```
admin> stats
Count: 3
Average: 90.0
Max: 95
Min: 85
```

학생 정보가 없을 경우 No student data available.

8.9 help

사용 가능한 명령어와 사용법을 출력한다.

사용법:

```
help
```

예:

Commands:	
save	Save students to CSV
reload	Reload students from CSV
add <id> <name> <score>	Add a student
delete <id>	Delete a student
update <id> <score>	Update student score
find <id>	Find student by ID
list	List all students
stats	Show statistics
clear	Clear screen
exit	Exit program

Client Program 에서는 Client 가 사용할 수 있는 명령어만 출력하거나, 제한된 명령어를 명확히 표시해야 한다.

Client Program 의 help 출력에는 save, add, delete, update 가 포함되지 않아야 하며, find 와 list 는 반드시 포함되어야 한다.

8.10 clear

터미널 화면을 지운다.

사용법:

```
clear
```

예:

```
printf("\033[2J\033[H");
```

간단한 ANSI escape sequence 를 사용할 수 있다.

8.11 exit

프로그램을 종료한다.

사용법:

```
exit
```

예:

```
admin> exit  
Goodbye.
```

종료하기 전에 동적으로 할당된 메모리를 모두 해제해야 한다.

9. 권장 명령어 처리 설계

명령어 처리는 함수 포인터 기반 command table 구조를 사용하는 것을 권장한다. 또한 Admin Program 과 Client Program 은 preprocessing flag 를 사용해 서로 다른 command table 을 구성할 수 있다.

```
typedef enum {  
    SHELL_OK = 0,  
    SHELL_EXIT,  
    SHELL_ERR_UNKNOWN_COMMAND,  
    SHELL_ERR_INVALID_ARGUMENT,  
    SHELL_ERR_MISSING_ARGUMENT,  
    SHELL_ERR_FILE_OPEN,  
    SHELL_ERR_FILE_WRITE,  
    SHELL_ERR_STUDENT_NOT_FOUND,  
    SHELL_ERR_DUPLICATE_STUDENT,  
    SHELL_ERR_INVALID_SCORE  
} ShellResult;  
  
typedef ShellResult (*CommandHandler)(char* args, Student** head);  
  
typedef struct {  
    const char* name;  
    CommandHandler handler;  
    const char* usage;  
    const char* description;  
} Command;
```

Admin Program 에서는 모든 명령어를 포함한다.

```
#ifdef ADMIN_MODE  
Command commands[] = {
```



```

    {"save", handle_save, "save", "Save students to CSV"},
    {"reload", handle_reload, "reload", "Reload students from CSV"},
    {"add", handle_add, "add <id> <name> <score>", "Add a student"},
    {"delete", handle_delete, "delete <id>", "Delete a student"},
    {"update", handle_update, "update <id> <score>", "Update student score"},
    {"find", handle_find, "find <id>", "Find student"},
    {"list", handle_list, "list", "List students"},
    {"stats", handle_stats, "stats", "Show statistics"},
    {"help", handle_help, "help", "Show help"},
    {"clear", handle_clear, "clear", "Clear screen"},
    {"exit", handle_exit, "exit", "Exit shell"}
};
#endif

```

Client Program 에서는 조회 명령어만 포함한다.

```

#ifdef CLIENT_MODE
Command commands[] = {
    {"reload", handle_reload, "reload", "Reload students from CSV"},
    {"find", handle_find, "find <id>", "Find student"},
    {"list", handle_list, "list", "List students"},
    {"stats", handle_stats, "stats", "Show statistics"},
    {"help", handle_help, "help", "Show help"},
    {"clear", handle_clear, "clear", "Clear screen"},
    {"exit", handle_exit, "exit", "Exit shell"}
};
#endif

```

10. 예외 처리 요구사항

프로그램은 사용자가 잘못된 입력을 하더라도 비정상 종료되지 않아야 한다.

명령어 오류

- 알 수 없는 명령어
- 인자 부족
- 잘못된 인자

데이터 오류

- 잘못된 ID
- 잘못된 이름
- 잘못된 점수
- 점수 범위 오류
- 중복된 학생 ID
- 존재하지 않는 학생
- 빈 목록

파일 오류

- CSV 파일 열기 실패

- CSV 파일 읽기 실패
- CSV 파일 쓰기 실패
- 잘못된 CSV 형식
- 잘못된 CSV header
- 명령어 파일 열기 실패

잘못된 CSV header 발생 시 "Error", "error", "invalid", "Invalid", 또는 "header" 중 하나를 포함하는 에러 메시지를 출력해야 한다.

10.1 명령어 파일 실행 중 오류 처리

명령어 파일을 실행하는 중 오류가 발생하면, 해당 줄의 오류 메시지를 출력하고 다음 명령어를 계속 실행해야 한다.

오류가 발생한 줄은 건너뛴다.

다음 줄의 명령어는 계속 실행한다.

```
[command file:2] add 4 David 88
Student added.
```

```
[command file:3] add 5 Jane abc
Error: invalid score. Skipped line 3.
```

```
[command file:4] list
ID   Name    Score
1    Alice    90
4    David    88
```

명령어 파일 자체를 열 수 없는 경우에는 명령어 파일 실행을 시작할 수 없으므로 오류 메시지를 출력하고 일반 interactive shell 로 진입하거나 프로그램을 종료할 수 있다. 이 동작은 README 에 명확히 설명해야 한다.

11. 권장 파일 구조

```
project/
├── main.c
├── student.h
├── student.c
├── file_io.h
├── file_io.c
├── command.h
├── command.c
├── Makefile
├── README.md
└── students.csv
```

파일	역할
main.c	프로그램 시작, 인자 처리, main loop
student.h	Student 구조체와 linked list 함수 선언
student.c	linked list 구현

file_io.h	CSV load/save 함수 선언
file_io.c	CSV 파일 입출력 구현
command.h	command 타입, command table, handler 선언
command.c	명령어 파싱과 command handler 구현
Makefile	빌드 자동화
README.md	프로젝트 설명과 사용법

12. Makefile 요구사항

프로젝트는 Makefile 을 제공해야 한다.

make admin 은 Admin Program 을 빌드하고, make client 는 Client Program 을 빌드해야 한다.

13. 실행 예시

예시 CSV 파일:

```
id,name,score
1,Alice,90
2,Bob,85
3,Charlie,95
```

Admin Program 실행:

```
./admin_shell students.csv
```

Client Program 실행:

```
./client_shell students.csv
```

Admin Program 실행 예:

```
[Admin Program]
Loaded 3 students from students.csv.
```

```
admin> list
ID   Name      Score
1    Alice     90
2    Bob       85
3    Charlie   95
```

```
admin> add 4 David 88
Student added.
```

```
admin> update 2 95
Student updated.
```

```
admin> delete 3
Student deleted.
```

```
admin> list
ID   Name      Score
```

```
1   Alice    90
2   Bob      95
4   David    88
```

```
admin> save
Saved 3 students to students.csv.
```

```
admin> exit
Goodbye.
```

명령어 파일 실행 예:

```
./admin_shell students.csv -f commands.txt
```

commands.txt:

```
list
add 4 David 88
update 99 70
find 4
save
exit
```

실행 결과 예:

```
[Admin Program]
Loaded 3 students from students.csv.
```

```
[command file:1] list
ID   Name      Score
1    Alice     90
2    Bob       85
3    Charlie   95
```

```
[command file:2] add 4 David 88
Student added.
```

```
[command file:3] update 99 70
Error: student not found. Skipped line 3.
```

```
[command file:4] find 4
ID: 4
Name: David
Score: 88
```

```
[command file:5] save
Saved 4 students to students.csv.
```

```
[command file:6] exit
Goodbye.
```

Client Program 실행 예:

```
[Client Program]
Loaded 3 students from students.csv.
```

```
client> list
ID   Name      Score
1    Alice     90
2    Bob       85
3    Charlie   95
```

```
client> add 4 David 88
Unknown command or permission denied.
```

```
client> exit  
Goodbye.
```

저장된 CSV 결과:

```
id,name,score  
1,Alice,90  
2,Bob,95  
4,David,88
```

14. 보고서 요구사항

소스 코드와 함께 보고서를 제출해야 한다.

1. 프로젝트 개요
2. 프로그램 구조
3. 자료구조 설계
4. CSV 파일 형식
5. 명령어 명세
6. Admin/Client Program 설계
7. 예외 처리 설계
8. 테스트 케이스와 결과
9. 한계점과 개선 방향

보고서는 프로그램이 어떻게 설계되고 구현되었는지 설명하는 문서여야 한다.

15. 채점 기준

총점: 100 점

15.1 구현 점수: 70 점

항목	배점	평가 내용
프로그램 실행 및 파일 인자 처리	5	CSV 파일명을 command-line argument 로 받고, -f 명령어 파일 옵션과 잘못된 실행 형식을 처리하는가
CSV File I/O	10	시작 시 CSV 를 읽고, save 로 저장하며, Excel 에서 확인 가능한 CSV 형식을 유지하는가
Linked List 구현	15	삽입, 삭제, 검색, 순회, 메모리 해제가 올바르게 동작하는가
Command 기능 구현	15	add, delete, update, find, list, stats, save, reload, help, clear, exit 등이 정상 동작하는가
Admin/Client Program	10	make admin, make client 로 두 실행 파일을 만들고, Admin 은 전체 기능을, Client 는 조회 전용 기능만 사용할 수 있는가
예외 처리	10	잘못된 명령어, 잘못된 인자, 파일 오류, CSV 오류, 중복 ID, 없는 학생, 명령어 파일 실행 중 오류, 메모리 오류 등을 안전하게 처리하는가
코드 구조 및 모듈화	5	.h/.c 파일 분리, 함수 역할 분리, 코드 가독성
Makefile 작성	5	make admin, make client, make clean 이 정상 동작하고 preprocessing flag 를 이용해 Admin/Client Program 을 각각 빌드하는가

15.2 보고서 점수: 30 점

항목	배점	평가 내용
설계 문서	10	전체 프로그램 구조, 모듈 역할, 데이터 흐름, Admin/Client Program 설계를 명확히 설명했는가
자료구조 설명	5	Student 구조체, linked list 동작, 메모리 관리 방식을 설명했는가
파일 입출력 설명	5	CSV 형식, load/save 과정, 파일 예시를 설명했는가
명령어 명세	5	각 명령어의 사용법, 권한, 예외 상황을 정리했는가
테스트 결과 및 실행 예시	5	실행 예시, 테스트 케이스, 출력 결과, 예외처리 테스트를 포함했는가

보고서 점수는 총 30 점이다.

15.3 제출 방식 및 주의사항

GitHub Classroom 제출

- GitHub Classroom 을 통해 중간 작성 결과를 10 회 이상 commit 하여 제출한다.
- 중간 commit 내역은 학생이 직접 코드를 작성했음을 판단하는 기준으로 사용될 수 있다.
- 각 commit 시 해당 시점의 완성 내용을 반드시 저장하여야 한다.

AI 활용 금지

- ChatGPT, Claude, Gemini 와 같은 generative AI 를 활용한 코딩은 금지한다.
- 불가피하게 AI 를 사용한 경우, AI 를 이용해 개발한 부분을 보고서에 명시하여야 한다.

기능 동작 확인

- 제출 전 기능 동작 여부는 기본으로 제공되는 grader.py 를 실행하여 확인한다.
- 최종 채점에서 사용하는 test case 는 변경될 수 있다.

16. 보너스 기능

보너스 기능은 선택 사항이다.

기능	보너스
-f <command_file> 옵션으로 파일에 저장된 명령어 실행	+3
명령어 파일에서 빈 줄과 # 주석 처리	+1
sort name 또는 sort score 구현	+2
저장하지 않은 변경사항이 있을 때 exit 시 경고	+1
메모리 누수 검사 결과 제출	+1

보너스 점수의 최대 반영 범위는 수업 운영 정책에 따라 제한될 수 있다.

sort 명령어 출력 요구사항:

sort name → "sorted by name" 을 포함하는 메시지 출력 필수

sort score → 점수 오름차순 정렬 (메시지 형식 자유, 결과 CSV 정렬 상태 필수)

sort <잘못된 키> → "Error" 를 포함하는 메시지 출력 필수

17. 제출 요구사항

다음 파일들을 제출해야 한다.

소스 파일: .c, .h

Makefile

README.md

프로젝트 보고서

샘플 CSV 파일

샘플 명령어 파일